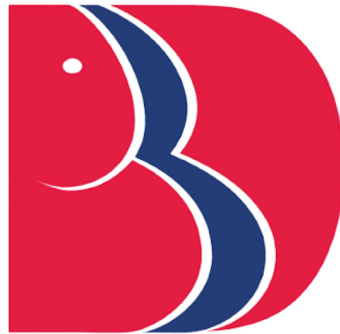


Babu Banarasi Das University



Case Study ON

Blinkit Grocery Data Analysis using Hadoop MapReduce and Hive

Submitted To:-

Mr. Vikash (IBM)

Submitted By:-

Name – Group E(Vishnu Tripathi)

Section – BCADS25

PROJECT

Blinkit Grocery Data Analysis using Hadoop MapReduce and Hive

➤ Architecture:

- The architecture of this project follows a standard Big Data pipeline using Hadoop ecosystem components. It shows how data flows from storage to processing and analysis to generate useful insights.

➤ Data Flow:

- Raw Dataset → HDFS → MapReduce Processing → Processed Data → Hive Analysis → Insights

➤ Explanation:

- Raw dataset (movie data) is first stored in HDFS
- MapReduce is used to process the data and calculate average ratings
- Processed data is stored back in HDFS
- Hive is used to perform analytical queries
- Final insights are generated from the results

➤ Tools Used:

- Hadoop Distributed File System (HDFS)
- MapReduce (Hadoop Streaming using Python)
- Hive (for data analysis)
- Linux Terminal
- ChatGPT (Generative AI tool)
- CSV Dataset

➤ Technologies:

- Python (for mapper and reducer)
- CSV dataset

- Hadoop Ecosystem

➤ **Use of Generative AI in the Project:**

- In this project, Generative AI tools were used to assist in development and understanding of Big Data concepts.

➤ **Areas Where AI Was Used:**

1. Code Generation

- Generative AI was used to generate MapReduce programs (mapper and reducer).

2. Debugging

- Errors in code were identified and fixed using AI assistance.

3. Query Writing

- Hive queries were generated and optimized using AI.

4. Concept Understanding

- AI helped in understanding:
 - HDFS architecture
 - MapReduce workflow
 - Hive operations

➤ **Example Prompt Used:**

- Write a Python MapReduce program to calculate average movie ratings.

➤ **Benefits of Using AI:**

- Faster development
- Reduced coding errors
- Better understanding of concepts
- Improved efficiency

➤ **Validation:**

- All AI-generated code was manually tested and verified before execution.

➤ **Dataset Description:**

- The dataset used in this project is a BlinkIT Grocery dataset containing information about grocery items and outlets. The data is stored in CSV (Comma Separated Values) format and is used for analysis using Hadoop MapReduce and Hive.

➤ **Dataset Structure:**

- The dataset consists of the following columns:
 - Item_Identifier – Unique ID of item
 - Item_Weight – Weight of item
 - Item_Fat_Content – Fat content type
 - Item_Visibility – Visibility of item in store
 - Item_Type – Category of item
 - Item_MRP – Price of item
 - Outlet_Identifier – Unique ID of outlet
 - Outlet_Establishment_Year – Year outlet was established
 - Outlet_Size – Size of outlet
 - Outlet_Location_Type – Location category of outlet

➤ **Benefits:**

- Helps understand Hadoop and Big Data processing
- Provides hands-on experience with MapReduce
- Improves knowledge of Hive queries
- Demonstrates real-world data analysis workflow
- Enhances practical skills in data handling

➤ **Purpose of Dataset:**

- **The dataset is used to:**
 - **Analyze item prices**
 - **Calculate average price by item type**
 - **Perform data processing using MapReduce**
 - **Perform data analysis using Hive**

➤ **Conclusion:**

- **In this practical, a complete Big Data processing pipeline was successfully implemented using the Hadoop ecosystem. The BlinkIT grocery dataset was stored in HDFS, processed using MapReduce to calculate average item prices, and analyzed using Hive to generate meaningful insights such as average price by item type and outlet analysis.**
- **The project helped in understanding the working of distributed data processing, Hadoop Streaming, and Hive queries for large datasets. It also improved practical knowledge of handling CSV data in a Big Data environment.**
- **Overall, this practical demonstrated how Hadoop and Hive can be used together to process and analyze data efficiently in real-world scenarios.**

❖ **Generative AI was used not only for code generation but also as a learning assistant to understand Big Data workflows and improve overall efficiency.**

➤ Question-1:- Item Price Analysis using Hadoop Streaming (MapReduce)

❖ **Aim: - “To analyze BlinkIT grocery dataset and calculate average Item_MRP (price) based on Item_Type using Hadoop MapReduce.”**

Step-01: - Create a “CSV” file.

- **Cat > blinkit_dataset.csv**

```
[cloudera@quickstart sf_Downloads]$ cat blinkit_dataset.csv
Item_Identifier,Item_Weight,Item_Fat_Content,Item_Visibility,Item_Type,Item_MRP,Outlet_Identifier,Outlet_Establishment_Year,Outlet_Size,Outlet_Location_Type
FDA15,9.3,Low Fat,0.016047301,Dairy,249.8092,OUT049,1999,Medium,Tier 1
DRC01,5.92,Regular,0.019278216,Soft Drinks,48.2692,OUT018,2009,Medium,Tier 3
FDN15,17.5,Low Fat,0.016760075,Meat,141.618,OUT049,1999,Medium,Tier 1
FDX07,19.2,Regular,0.0,Fruits and Vegetables,182.095,OUT010,1998,,Tier 3
NCD19,8.93,Low Fat,0.0,Household,53.8614,OUT013,1987,High,Tier 3
FDX36,10.395,Regular,0.0,Baking Goods,51.4008,OUT018,2009,Medium,Tier 3
FD010,13.65,Regular,0.012741089,Snack Foods,57.6588,OUT013,1987,High,Tier 3
FDP10,,Low Fat,0.127469857,Snack Foods,107.7622,OUT027,1985,Medium,Tier 3
FDH17,16.2,Regular,0.016687114,Frozen Foods,96.9726,OUT045,2002,,Tier 2
FDU28,19.2,Regular,0.09444959,Frozen Foods,187.8214,OUT017,2007,,Tier 2
FDY07,11.8,Low Fat,0.0,Fruits and Vegetables,45.5402,OUT049,1999,Medium,Tier 1
FDA03,18.5,Regular,0.045463773,Dairy,144.1102,OUT046,1997,Small,Tier 1
FDX32,15.1,Regular,0.1000135,Fruits and Vegetables,145.4786,OUT049,1999,Medium,Tier 1
FDS46,17.6,Regular,0.047257328,Snack Foods,119.6782,OUT046,1997,Small,Tier 1
FDF32,16.35,Low Fat,0.0680243,Fruits and Vegetables,196.4426,OUT013,1987,High,Tier 3
FDP49,9.0,Regular,0.069088961,Breakfast,56.3614,OUT046,1997,Small,Tier 1
NCB42,11.8,Low Fat,0.008596051,Health and Hygiene,115.3492,OUT018,2009,Medium,Tier 3
FDP49,9.0,Regular,0.069196376,Breakfast,54.3614,OUT049,1999,Medium,Tier 1
DRI11,,Low Fat,0.034237682,Hard Drinks,113.2834,OUT027,1985,Medium,Tier 3
FDU02,13.35,Low Fat,0.10249212,Dairy,230.5352,OUT035,2004,Small,Tier 2
FDN22,18.85,Regular,0.138190277,Snack Foods,250.8724,OUT013,1987,High,Tier 3
FDW12,,Regular,0.035399923,Baking Goods,144.5444,OUT027,1985,Medium,Tier 3
NCB30,14.6,Low Fat,0.025698134,Household,196.5084,OUT035,2004,Small,Tier 2
FDC37,,Low Fat,0.057556998,Baking Goods,107.6938,OUT019,1985,Small,Tier 1
FDR28,13.85,Regular,0.025896485,Frozen Foods,165.021,OUT046,1997,Small,Tier 1
NCD06,13.0,Low Fat,0.099887103,Household,45.906,OUT017,2007,,Tier 2
FDV10,7.645,Regular,0.066693437,Snack Foods,42.3112,OUT035,2004,Small,Tier 2
DRJ59,11.65,Low fat,0.019356132,Hard Drinks,39.1164,OUT013,1987,High,Tier 3
FDE51,5.925,Regular,0.161466534,Dairy,45.5086,OUT010,1998,,Tier 3
FDC14,,Regular,0.072221801,Canned,43.6454,OUT019,1985,Small,Tier 1
FDV38,19.25,Low Fat,0.170348551,Dairy,55.7956,OUT010,1998,,Tier 3
NCS17,18.6,Low Fat,0.080829372,Health and Hygiene,96.4436,OUT018,2009,Medium,Tier 3
FDP33,18.7,Low Fat,0.0,Snack Foods,256.6672,OUT018,2009,Medium,Tier 3
```

Step-02:- Create a directory

- **hdfs dfs -mkdir /user/cloudera/Vishnu**
- **hdfs dfs -mkdir /user/cloudera/Vishnu/Blinkit**

```
[cloudera@quickstart sf_Downloads]$ hdfs dfs -mkdir /user/cloudera/Vishnu
[cloudera@quickstart sf_Downloads]$ hdfs dfs -mkdir /user/cloudera/Vishnu/Blinkit
[cloudera@quickstart sf_Downloads]$ hdfs dfs -ls /user/cloudera/Vishnu/Blinkit
[cloudera@quickstart sf_Downloads]$ hdfs dfs -ls /user/cloudera/Vishnu
Found 1 items
drwxr-xr-x - cloudera cloudera          0 2026-04-24 22:44 /user/cloudera/Vishnu/Blinkit
[cloudera@quickstart sf_Downloads]$
```

Step-03:- Load the “CSV” on Cloudera’s Server

- **hdfs dfs -put blinkit_dataset.csv /user/cloudera/Vishnu/Blinkit**
- **hdfs dfs -ls /user/cloudera/Vishnu/Blinkit**

```
[cloudera@quickstart sf_Downloads]$ hdfs dfs -put blinkit_dataset.csv /user/cloudera/Vishnu/Blinkit
[cloudera@quickstart sf_Downloads]$ hdfs dfs -ls /user/cloudera/Vishnu/Blinkit
Found 1 items
-rw-r--r-- 1 cloudera cloudera      3136 2026-04-24 22:47 /user/cloudera/Vishnu/Blinkit/blinkit_dataset.csv
[cloudera@quickstart sf_Downloads]$
```

nanStep-04:- Create a mapper file using “nano”

- **nano mapper.py**

```
#!/usr/bin/env python
import sys
```

```
for line in sys.stdin:
    if "Item_Identifier" in line:
        continue
```

```
    data = line.strip().split(',')
    item_type = data[4]
    item_mrp = float(data[5])
```

```
    print("%s\t%s" % (item_type, item_mrp))
```

```
#!/usr/bin/env python
import sys

for line in sys.stdin:
    if "Item_Identifier" in line:
        continue

    data = line.strip().split(',')

    item_type = data[4]
    item_mrp = float(data[5])

    print("%s\t%s" % (item_type, item_mrp))
```

Step-05:- Create a reducer file using “nano”

- nano reducer.py
- `#!/usr/bin/env python`
- `import sys`
-
- `current_type = None`
- `total = 0`
- `count = 0`
-
- `for line in sys.stdin:`
- `item_type, item_mrp = line.strip().split('\t')`
- `item_mrp = float(item_mrp)`
-
- `if current_type == item_type:`
- `total += item_mrp`
- `count += 1`
-
- `else:`
- `if current_type:`
- `print("%s\t%s" % (current_type, total/count))`
-
- `current_type = item_type`
- `total = item_mrp`
- `count = 1`
-
- `if current_type:`
- `print("%s\t%s" % (current_type, total/count))`

```
#!/usr/bin/env python
import sys

current_type = None
total = 0
count = 0

for line in sys.stdin:
    item_type, item_mrp = line.strip().split('\t')
    item_mrp = float(item_mrp)

    if current_type == item_type:
        total += item_mrp
        count += 1

    else:
        if current_type:
            print("%s\t%s" % (current_type, total/count))

        current_type = item_type
        total = item_mrp
        count = 1

if current_type:
    print("%s\t%s" % (current_type, total/count))
```


Step-06:- Make executable

- **chmod +x mapper.py reducer.py**

```
[cloudera@quickstart sf_Downloads]$ chmod +x mapper.py reducer.py
[cloudera@quickstart sf_Downloads]$ ls
archive
archive (1)
archive (1).zip
archive (2)
archive (2).zip
archive.zip
arsalan_resume.pdf
bbdu
bd module 1.pdf
BD_PROJECT_NEW (2) (1).docx
[cloudera@quickstart sf_Downloads]$ ls -l
total 4586386
drwxrwx--- 1 root vboxsf 0 Apr 16 23:07 archive
drwxrwx--- 1 root vboxsf 0 Apr 18 04:55 archive (1)
-rwxrwx--- 1 root vboxsf 3144815 Apr 18 04:55 archive (1).zip
drwxrwx--- 1 root vboxsf 4096 Apr 24 22:11 archive (2)
-rwxrwx--- 1 root vboxsf 695160 Apr 24 22:10 archive (2).zip
-rwxrwx--- 1 root vboxsf 1599 Apr 16 23:06 archive.zip
-rwxrwx--- 1 root vboxsf 18442 Apr 21 04:38 arsalan_resume.pdf
drwxrwx--- 1 root vboxsf 8192 Apr 20 04:37 bbdu
-rwxrwx--- 1 root vboxsf 1212123 Apr 17 00:20 bd module 1.pdf
-rwxrwx--- 1 root vboxsf 1065884 Apr 22 02:45 BD_PROJECT_NEW (2) (1).docx
mapper.py
Microsoft.Services.Store.winmd
python-3.13.13-amd64.exe
reducer.py
SpotifySetup.exe
student-performance-dataset.ipynb
VC_redist.x64.exe
VirtualBox-7.2.6a-172322-Win.exe
VSCoUserSetup-x64-1.115.0.exe
WhatsApp_Installer.exe
```

Step-07:- Local test

- **cat movie.csv | python mapper.py**
- **cat movie.csv | python mapper.py | sort | python reducer.py**

```
[cloudera@quickstart sf_Downloads]$ cat blinkit_dataset.csv | python mapper.py
Dairy 249.8092
Soft Drinks 48.2692
Meat 141.618
Fruits and Vegetables 182.095
Household 53.8614
Baking Goods 51.4008
Snack Foods 57.6588
Snack Foods 107.7622
Frozen Foods 96.9726
Frozen Foods 187.8214
Fruits and Vegetables 45.5402
Dairy 144.1102
Fruits and Vegetables 145.4786
Snack Foods 119.6782
Fruits and Vegetables 196.4426
Breakfast 56.3614
Health and Hygiene 115.3492
Breakfast 54.3614
Hard Drinks 113.2834
Dairy 230.5352
Snack Foods 250.8724
Baking Goods 144.5444
Household 196.5084
Baking Goods 107.6938
Frozen Foods 165.021
Household 45.906
Snack Foods 42.3112
Hard Drinks 39.1164
Dairy 45.5086
Canned 43.6454
Dairy 55.7956
Health and Hygiene 96.4436
Snack Foods 256.6672
Breads 93.1436
Soft Drinks 174.8738
Snack Foods 87.6198
[cloudera@quickstart sf_Downloads]$ cat blinkit_dataset.csv | python mapper.py | sort | python reducer.py
Baking Goods 101.213
Breads 93.1436
Breakfast 55.3614
Canned 43.6454
Dairy 145.15176
Frozen Foods 149.938333333
Fruits and Vegetables 139.52484
Hard Drinks 76.1999
Health and Hygiene 119.501
Household 98.7586
Meat 141.618
Snack Foods 119.94465
Soft Drinks 115.227266667
[cloudera@quickstart sf_Downloads]$
```

Step-08:- Run Hadoop Job

- **hadoop jar /usr/lib/hadoop-mapreduce/hadoop-streaming.jar -files mapper.py,reducer.py -input /user/cloudera/Vishnu/Blinkit/blinkit.dataset.csv -output /user/cloudera/Vishnu/Blinkit/output -mapper "python mapper.py" -reducer "python reducer.py"**

```
[cloudera@quickstart sf_Downloads]$ hadoop jar /usr/lib/hadoop-mapreduce/hadoop-streaming.jar -files mapper.py, reducer.py
put /user/cloudera/Vishnu/Blinkit/blinkit_dataset.csv -output /user/cloudera/Vishnu/Blinkit/output -mapper "python mapper.py"
-reducer "python reducer.py"
packageJobJar: [] [/usr/jars/hadoop-streaming-2.6.0-cdh5.4.2.jar] /tmp/streamjob2517628318733512532.jar tmpDir=null
26/04/24 23:15:10 INFO client.RMProxy: Connecting to ResourceManager at /0.0.0.0:8032
26/04/24 23:15:10 INFO client.RMProxy: Connecting to ResourceManager at /0.0.0.0:8032
26/04/24 23:15:11 INFO mapred.FileInputFormat: Total input paths to process : 1
26/04/24 23:15:11 INFO mapreduce.JobSubmitter: number of splits:2
26/04/24 23:15:11 INFO mapreduce.JobSubmitter: Submitting tokens for job: job_1777094353749_0001
26/04/24 23:15:12 INFO impl.YarnClientImpl: Submitted application application_1777094353749_0001
26/04/24 23:15:12 INFO mapreduce.Job: The url to track the job: http://quickstart.cloudera:8088/proxy/application_1777094353749_0001/
26/04/24 23:15:12 INFO mapreduce.Job: Running job: job_1777094353749_0001
26/04/24 23:15:22 INFO mapreduce.Job: Job job_1777094353749_0001 running in uber mode : false
26/04/24 23:15:22 INFO mapreduce.Job: map 0% reduce 0%
26/04/24 23:15:34 INFO mapreduce.Job: map 100% reduce 0%
26/04/24 23:15:44 INFO mapreduce.Job: map 100% reduce 100%
26/04/24 23:15:44 INFO mapreduce.Job: Job job_1777094353749_0001 completed successfully
26/04/24 23:15:44 INFO mapreduce.Job: Counters: 49
```

Step-9:- View Output

- **hdfs dfs -cat /user/cloudera/Vishnu/Blinkit/output/part-00000**

```
[cloudera@quickstart sf_Downloads]$ hdfs dfs -cat /user/cloudera/Vishnu/Blinkit/output/part-00000
Baking Goods      101.213
Breads            93.1436
Breakfast         55.3614
Canned            43.6454
Dairy             145.15176
Frozen Foods      149.938333333
Fruits and Vegetables 139.52484
Hard Drinks       76.1999
Health and Hygiene 119.501
Household         98.7586
Meat              141.618
Snack Foods       119.94465
Soft Drinks       115.227266667
[cloudera@quickstart sf_Downloads]$
```

➤ Question-2:- Outlet Analysis using Hive.

- ❖ **Aim: "To analyze BlinkIT dataset and find average Item_MRP by Outlet_Location_Type using Hive queries."**

Step-01:- Open Hive

- **hive**

```
[cloudera@quickstart mapreduce]$ hive

Logging initialized using configuration in file:/etc/hive/conf.dist/hive-log4j.properties
WARNING: Hive CLI is deprecated and migration to Beeline is recommended.
hive>
```

Step-02:- Create table on hive

- **CREATE TABLE blinkit_data (**
 Item_Identifier STRING,
 Item_Weight FLOAT,
 Item_Fat_Content STRING,
 Item_Visibility FLOAT,
 Item_Type STRING,
 Item_MRP FLOAT,
 Outlet_Identifier STRING,
 Outlet_Establishment_Year INT,
 Outlet_Size STRING,
 Outlet_Location_Type STRING
)
 ROW FORMAT DELIMITED
 FIELDS TERMINATED BY ','
 STORED AS TEXTFILE;

```
hive> CREATE TABLE blinkit_data (  
  > Item_Identifier STRING,  
  > Item_Weight FLOAT,  
  > Item_Fat_Content STRING,  
  > Item_Visibility FLOAT,  
  > Item_Type STRING,  
  > Item_MRP FLOAT,  
  > Outlet_Identifier STRING,  
  > Outlet_Establishment_Year INT,  
  > Outlet_Size STRING,  
  > Outlet_Location_Type STRING  
  > )  
  > ROW FORMAT DELIMITED  
  > FIELDS TERMINATED BY ','  
  > STORED AS TEXTFILE;
```

OK

Time taken: 3.489 seconds

hive> █

Step-03:- Load data on the table in movie_avg

- **LOAD DATA INPATH**
 '/user/cloudera/Vishnu/Blinkit/blinkit_dataset.csv'

- **INTO TABLE blinkit_data;**

```
hive> LOAD DATA INPATH '/user/cloudera/Vishnu/Blinkit/blinkit_dataset.csv'
> INTO TABLE blinkit_data;
Loading data to table default.blinkit_data
chgrp: changing ownership of 'hdfs://quickstart.cloudera:8020/user/hive/warehouse/blinkit_data/blinkit_dataset.csv': Use
s not belong to hive
Table default.blinkit_data stats: [numFiles=1, totalSize=3136]
OK
Time taken: 1.228 seconds
hive>
```

Step-04:- View Data

- **Select * from blinkit_dataset;**

```
hive> SELECT * FROM blinkit_data;
OK
Item_Identifier NULL Item_Fat_Content NULL Item_Type NULL Outlet_Identifier NULL Outl
utlet_Location_Type
FDA15 9.3 Low Fat 0.0160473 Dairy 249.8092 OUT049 1999 Medium Tier 1
DRC01 5.92 Regular 0.019278215 Soft Drinks 48.2692 OUT018 2009 Medium Tier 3
FDN15 17.5 Low Fat 0.016760075 Meat 141.618 OUT049 1999 Medium Tier 1
FDX07 19.2 Regular 0.0 Fruits and Vegetables 182.095 OUT010 1998 Tier 3
NCD19 8.93 Low Fat 0.0 Household 53.8614 OUT013 1987 High Tier 3
FDP36 10.395 Regular 0.0 Baking Goods 51.4008 OUT018 2009 Medium Tier 3
FD010 13.65 Regular 0.012741089 Snack Foods 57.6588 OUT013 1987 High Tier 3
FDP10 NULL Low Fat 0.12746985 Snack Foods 107.7622 OUT027 1985 Medium Tier 3
FDH17 16.2 Regular 0.016687114 Frozen Foods 96.9726 OUT045 2002 Tier 2
FDU28 19.2 Regular 0.09444959 Frozen Foods 187.8214 OUT017 2007 Tier 2
FDY07 11.8 Low Fat 0.0 Fruits and Vegetables 45.5402 OUT049 1999 Medium Tier 1
FDA03 18.5 Regular 0.045463774 Dairy 144.1102 OUT046 1997 Small Tier 1
FDX32 15.1 Regular 0.1000135 Fruits and Vegetables 145.4786 OUT049 1999 Medium Tier 1
FDS46 17.6 Regular 0.047257327 Snack Foods 119.6782 OUT046 1997 Small Tier 1
FDP32 16.35 Low Fat 0.0680243 Fruits and Vegetables 196.4426 OUT013 1987 High Tier 3
FDP49 9.0 Regular 0.06908896 Breakfast 56.3614 OUT046 1997 Small Tier 1
NCB42 11.8 Low Fat 0.008596051 Health and Hygiene 115.3492 OUT018 2009 Medium Tier 3
FDP49 9.0 Regular 0.06919637 Breakfast 54.3614 OUT049 1999 Medium Tier 1
DRI11 NULL Low Fat 0.034237683 Hard Drinks 113.2834 OUT027 1985 Medium Tier 3
FDU02 13.35 Low Fat 0.10249212 Dairy 230.5352 OUT035 2004 Small Tier 2
FDN22 18.85 Regular 0.13819028 Snack Foods 250.8724 OUT013 1987 High Tier 3
FDN12 NULL Regular 0.03539992 Baking Goods 144.5444 OUT027 1985 Medium Tier 3
NCB30 14.6 Low Fat 0.025698135 Household 196.5084 OUT035 2004 Small Tier 2
FDC37 NULL Low Fat 0.057556998 Baking Goods 107.6938 OUT019 1985 Small Tier 1
```

Step-05:- Find Average Price by Location

- **SELECT Outlet_Location_Type,**

- **AVG(Item_MRP)**
- **FROM blinkit_data**
- **GROUP BY Outlet_Location_Type;**

```
hive> SELECT Outlet_Location_Type,
> AVG(Item_MRP)
> FROM blinkit_data
> GROUP BY Outlet_Location_Type;
Query ID = cloudera_20260424233131_d98f09e0-26fc-492c-a015-10d28487ceb1
Total jobs = 1
Launching Job 1 out of 1
Number of reduce tasks not specified. Estimated from input data size: 1
In order to change the average load for a reducer (in bytes):
  set hive.exec.reducers.bytes.per.reducer=<number>
In order to limit the maximum number of reducers:
  set hive.exec.reducers.max=<number>
In order to set a constant number of reducers:
  set mapreduce.job.reduces=<number>
Starting Job = job_1777094353749_0002, Tracking URL = http://quickstart.cloudera:8088/proxy/application_1777094353749_0002
Kill Command = /usr/lib/hadoop/bin/hadoop job -kill job_1777094353749_0002
Hadoop job information for Stage-1: number of mappers: 1; number of reducers: 1
2026-04-24 23:31:53,813 Stage-1 map = 0%, reduce = 0%
2026-04-24 23:32:06,211 Stage-1 map = 100%, reduce = 0%, Cumulative CPU 1.94 sec
2026-04-24 23:32:16,956 Stage-1 map = 100%, reduce = 100%, Cumulative CPU 4.54 sec
MapReduce Total cumulative CPU time: 4 seconds 540 msec
Ended Job = job_1777094353749_0002
MapReduce Jobs Launched:
Stage-Stage-1: Map: 1 Reduce: 1 Cumulative CPU: 4.54 sec HDFS Read: 11747 HDFS Write: 102 SUCCESS
Total MapReduce CPU Time Spent: 4 seconds 540 msec
OK
Outlet_Location_Type    NULL
Tier 1  122.68472348726712
Tier 2  127.59977231706891
Tier 3  109.51423034667968
Time taken: 35.509 seconds, Fetched: 4 row(s)
hive> █
```

➤ **Output:**

- **Hive successfully displayed:**
 - Average Item Price (Item_MRP)
 - Top-selling Item Types
 - Outlet-wise Item Analysis

❖ **In this practical, Hive was used to analyze processed movie data and retrieve meaningful insights using SQL-like queries.**